Name:

MIDTERM | MPCS 52060 – PARALLEL PROGRAMMING | FEB 02, 2023

This document consists of 15 numbered pages, including this one. In total, problems are worth 100 points.

We will ONLY grade solutions on the numbered pages!

We will only grade pages that have your name at the top!

We will provide scratch paper during the test on request. Please note that we will not grade anything written on scratch paper. All your responses must appear on this copy of the test itself. Please only write on the front of each page.

Write helper functions as you see fit. You may use any function you write anywhere on the exam.

If you believe you have found an error in the test (typo or otherwise), let me know; if there is an error, we will announce it to the whole class and write a note on the board. We will not clarify, confirm the meaning of test questions, or answer questions for individuals, though, since this gives students who ask questions an advantage over those who don't.

Problem	**Maximum Points**	**Score**
Short Answer Questions	65	
Programming Questions	35	

SHORT ANSWER QUESTIONS

We allow plenty of space, but this does not mean that you need to provide long answers. Provide a brief answer to the following questions.

SAQ 1 (2 POINTS)

In your assignments you created a plot for speedup as a function of the number of threads. What is the formula for speedup? Assume that N is the number of threads, T_0 is the run time with one thread, T_N is the run time with N threads, and S is the speedup. Assume that the run time of the sequential program is the same as that of the parallel program with 1 thread.

SAQ 2 (3 POINTS)

What does it mean to have a fractional speedup (e.g. 0.5)?

SAQ 3 (5 POINTS)

Suppose a program has two portions. When executed with 1 thread, Portion A takes 25% of the overall run time. Portion A cannot be parallelized. The remaining portion B takes 75% of the overall run time when executed with 1 thread. Portion B can be parallelized perfectly, meaning that this portion runs N times faster with N threads than it does with 1 thread.

1. What is the overall speedup of this program when executed with 3 threads?

2. What is the maximum overall speedup when executed with an infinitely large number of threads?

SAQ 4 (5 POINTS)

Why are TTAS locks faster than TAS locks? Briefly summarize the events that cause the slowdown in TAS.

SAQ 5 (5 POINTS)

A parallel program spawns multiple threads that are accessing entries in the same shared array.

1. Give an example of a situation where most of these accesses would result in a cache hit. Describe the access patterns (which thread accesses which indices) that would cause this, and briefly explain why.

2. Give an example of a situation where most of these accesses would result in a cache miss. Describe the access patterns (which thread accesses which indices) that would cause this, and briefly explain why.

SAQ 6 (10 POINTS)

1. Name one data structure that we discussed in class, which can be implemented with either fine-grained or coarse-grained locking strategies.

2. For both strategies, list the operations we discussed for this data structure, and state for each operation whether it requires acquiring one or more locks, and if so, **how many locks need to be acquired**. It is not necessary to write code here.

3. For both strategies, explain **how many locks need to be created**, for example as a function of the size of the data structure or problem size, and the number of threads (define reasonable variables as needed). Count every lock, regardless of whether it is currently being held or not.

SAQ 7 (10 POINTS)

During class we discussed safety properties.

1. Describe an algorithm with a deadlock. You may use pseudo-code. Describe the sequence of events that would cause the deadlock to occur.

2. Describe an algorithm with a race condition. You may use pseudo-code. Describe the sequence of events that would cause the race condition to occur.

SAQ 8 (10 POINTS)

During class we discussed fairness.

1. What is the fairness principle?

2. Give two examples of locks that we discussed during class, which violate the fairness property. Briefly explain why for each case (if the explanation is the same, you may simply say so).

3. Give an example of two locks that we discussed during class, which obey the fairness principle. Briefly explain why for each case (if the explanation is the same, you may simply say so).

SAQ 9 (10 POINTS)

Review the following program:

```
package main

import "sync"
import "fmt"

func modifyVal(s []int, wg *sync.WaitGroup) {
    s[0] = s[0] + 1
    fmt.Println("inside: ", s)
    wg.Done()
}

func main() {
    var wg sync.WaitGroup

    s := make([]int, 2)

    s[0] = 1
    s[1] = 2

    wg.Add(2)
    go modifyVal(s, &wg)
    go modifyVal(s, &wg)
    wg.Wait()
    fmt.Println("outside: ", s)
}
```

Use the following empty page to answer these questions:

1. Does this program have a data race? If so, where, and why?
2. What does this program print to the terminal? If there are multiple possibilities, list all of them.

Name:

(EMPTY PAGE FOR YOUR SAQ 9 ANSWER)

Name:

____

SAQ 10 (5 POINTS)

Terminology.

1. Explain the difference between concurrency and parallelism. Does one require the other?

2. Assume your computer has one processor with 4 cores, with no instruction-level parallelism or hyperthreading. Can it run more than 4 threads? Why or why not?

3. Briefly explain pipelining (1-2 sentences are enough).

4. Briefly explain the difference between latency and throughput (1-2 sentences are enough).

____

PROGRAMMING QUESTIONS

Answer the following programming questions. You need to write valid Go code that solves each problem.

We will ignore minor spelling/syntax errors if the intent is clear. For example, slightly misspelling a function name, forgetting to import a package that you are using, or forgetting the exact syntax for creating an array will probably not lead to deductions if the intent is clear. On the other hand, problems with the semantics (e.g., passing a variable where you should have passed its pointer, using a lock where you should have used a semaphore, creating an array with the wrong size, creating a slice that starts/ends at the wrong place, etc..) will lead to deductions, even if this happened just because of a typo/misspelling.

We provide both problems back-to-back, please use the remaining numbered pages as needed and clearly mark which part of your writeup belongs to which problem.

PQ1 (15 POINTS)

Implement a Go library (similar to the libraries you created for homework 3) that implements the Bakery algorithm presented in class. You will need to at least implement the following:

1. A method to initialize the lock with anything required by the algorithm (e.g., NewBakery(...)). You are allowed to pass in any additional arguments that make sense.
2. Methods to lock (Lock(...)) and unlock (Unlock(...)) the lock. You are allowed to pass in any additional arguments that make sense.

You can create any new data structures and helper functions necessary to implement the algorithm. If necessary, you can use sync/atomic operations in your implementation.

PQ2 (20 POINTS)

Imagine n gorountines, each of which executes function foo(...) followed by function bar(...). Suppose we want to make sure that no gorountine starts bar(...) until all gorountines have finished foo(...). Write a concurrent Go program that implements this scenario. The only output in each function are the threads stating that they are in either bar(...) or foo(...). For example, if n = 3 then the following is one valid output of the program:

```
Thread 3 is in foo
Thread 2 is in foo
Thread 1 is in foo
Thread 1 is in bar
Thread 3 is in bar
Thread 2 is in bar
```

The implementation must only use mutexes, condition variables, and/or atomic operations. Set the number of threads to be 8 (i.e., n = 8).

Name: ____

Name:

___

Name:

____